

Arquitectura y stack tecnologico - GEO ARTE CDMX

Version: 1.0 - Fecha: junio 2026 Proyecto: 'geoarte-web' - Plataforma: aplicacion web para inteligencia territorial cultural en CDMX

1. Resumen ejecutivo

GEO ARTE CDMX es una plataforma web para visualizar, analizar y gestionar la infraestructura cultural de las 16 alcaldias de la Ciudad de Mexico. Permite consultar un mapa territorial interactivo, KPIs y graficos estadisticos, generacion de reportes, repositorio de investigacion cualitativa, recomendaciones de politicas publicas y un panel de administracion para perfiles con rol Autoridad.

La aplicacion esta construida como un monolito full-stack con Next.js 16 (App Router), React 19, TypeScript y Supabase como backend (autenticacion, base de datos PostgreSQL y almacenamiento). El mapa usa Leaflet con capas GeoJSON y datos sincronizados desde Supabase.

2. Stack tecnologico

2.1 Frontend y framework

Herramienta	Version	Uso
Next.js	16.2.6	Framework full-stack: rutas, SSR, API Routes, Server Actions
React	19.2.4	Biblioteca de interfaz de usuario
TypeScript	5.x	Tipado estatico en todo el codigo fuente
Tailwind CSS	4.x	Estilos utilitarios y diseno responsivo
PostCSS	-	Procesamiento de CSS ('@tailwindcss/postcss')
Inter	(Google Fonts)	Tipografia principal via 'next/font'
Lucide React	1.16	Iconografia del sistema

2.2 Backend y datos

Herramienta	Version	Uso
Supabase	-	Backend as a Service: PostgreSQL, Auth, Storage, RLS
@supabase/supabase-js	2.106	Cliente JavaScript para consultas y mutaciones
@supabase/ssr	0.10	Sesiones SSR en middleware y servidor Next.js

2.3 Visualizacion y exportacion

Herramienta	Version	Uso
Leaflet	1.9.4	Mapa interactivo (alcaldias, macrozonas, transporte, espacios)
Recharts	3.8	Graficos del dashboard y secciones analiticas
jsPDF	4.2	Generacion de PDF en cliente y scripts de documentacion
JSZip	3.10	Empaquetado de archivos en exportaciones
xlsx	0.18	Exportacion de datos a Excel

2.4 Herramientas de desarrollo

Herramienta	Uso
ESLint	Linting con 'eslint-config-next'
Node.js	Scripts de seed, sincronizacion y utilidades ('scripts/')
npm	Gestor de paquetes y scripts del proyecto

2.5 Fuentes de datos geograficos

- src/data/geo/cdmx-alcaldias.geojson - poligonos de las 16 alcaldias
- src/data/geo/cdmx-metro-lineas.geojson - lineas del Metro
- src/data/geo/cdmx-metrobus-lineas.geojson - lineas del Metrobus

3. Arquitectura de software

La aplicacion sigue un patron Modelo - Vista - Controlador (MVC) adaptado a Next.js y React.

3.1 Capas

Capa | Ubicacion | Responsabilidad

****Modelo**** | 'src/lib/domain/', 'src/lib/data/' | Tipos de dominio, repositorios Supabase y datos mock

****Controlador**** | 'src/lib/services/', 'src/hooks/', 'src/actions/', '*Controller.tsx' | Orquestacion de datos, reglas de negocio y estado

****Vista**** | 'src/components/features/*View.tsx', 'shared/', 'layout/' | Presentacion JSX; recibe datos por props

3.2 Flujo por pantalla

```
app/[ruta]/page.tsx -> *Controller.tsx -> *.service.ts -> lib/data/supabase | mock
                        |
                        +-- *View.tsx (+ hooks si hay estado cliente)
```

- Las rutas en src/app/ solo enrutan y definen metadata SEO.
- Los servicios (src/lib/services/) son el punto unico para alternar entre datos mock y Supabase.
- Los hooks (src/hooks/) concentran estado interactivo (filtros, pestanas, formularios).
- Las Server Actions (src/actions/) gestionan mutaciones en servidor cuando aplica.

3.3 Modo demo vs. produccion

La aplicacion puede operar en dos modos:

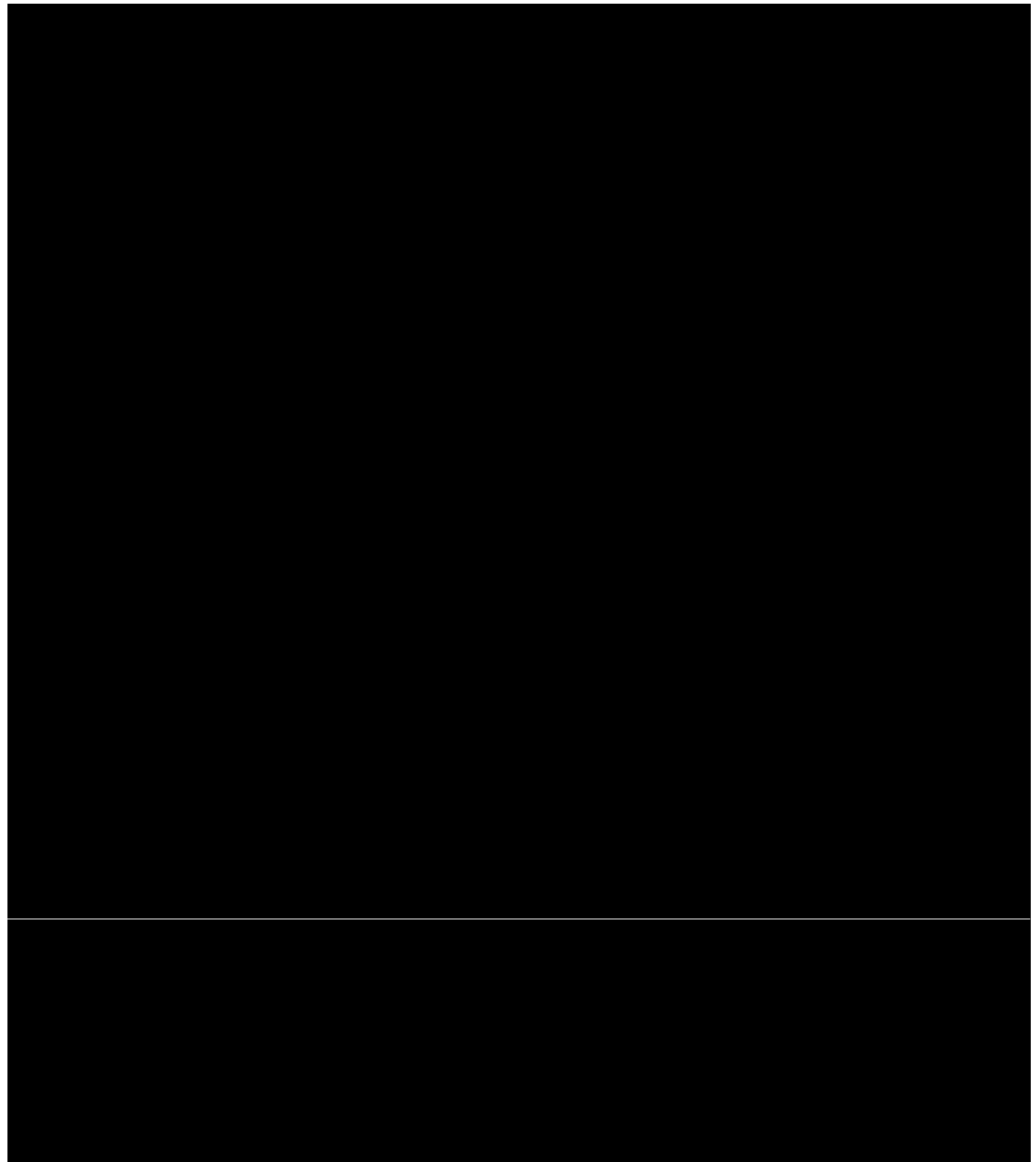
Modo | Condicion | Comportamiento

****Demo**** | Sin variables 'NEXT_PUBLIC_SUPABASE_*' | Datos de ejemplo desde 'src/lib/data/mock/'

****Supabase**** | Credenciales configuradas en '.env.local' | Datos reales del padron SECTEI, metricas y repositorio

Si Supabase esta configurado pero falla una consulta, muchas secciones degradan a modo demo con un aviso visual.

4. Estructura del proyecto



'/mapa' | Mapa territorial | Visualizacion GIS con capas y filtros
'/dashboard' | Dashboard | Graficos, comparadores y exportacion del padron
'/reportes' | Reportes | Plantillas configurables y descarga de informes
'/investigacion' | Investigacion | Recursos cualitativos (entrevistas, estudios, etc.)
'/politicas' | Politicas | Recomendaciones y evidencia para politica publica
'/contacto' | Soporte | Formulario, datasets y documentacion de API
'/admin' | Administracion | Gestion de espacios, usuarios, capas y configuracion
'/perfil' | Mi perfil | Datos personales, historial y recursos guardados

5.2 Autenticacion

Rutas de autenticacion gestionadas con Supabase Auth:

- /login - inicio de sesion
- /registro - alta de cuenta

- /verificar-email - confirmacion de correo
- /recuperar-contrasena - solicitud de restablecimiento
- /restablecer-contrasena - nueva contrasena
- /email-verificado - confirmacion post-verificacion

El middleware ('src/middleware.ts') renueva la sesion Supabase, redirige usuarios no autenticados y valida verificacion de correo antes de acceder a la aplicacion.

5.3 Perfiles de usuario (roles)

Rol en Supabase | Rol en app | Permisos destacados
 Ciudadano | 'ciudadano' | Consulta publica, perfil personal
 Investigador | 'investigador' | Igual que ciudadano + acceso a recursos de investigacion
 Autoridad | 'autoridad' | Panel Admin, sincronizacion de mapa, gestion de usuarios

La navegacion filtra el item Administracion para roles distintos de Autoridad.

6. API y endpoints

La aplicacion expone 48 rutas API organizadas en tres grupos:

6.1 API de datos interna ('/api/data/')

Alimenta las pantallas del frontend con JSON:

- /api/data/home - KPIs de inicio
- /api/data/dashboard - metricas y series del dashboard
- /api/data/mapa - capas y geometrias territoriales
- /api/data/mapa/transporte - lineas de transporte masivo
- /api/data/investigacion - listado de recursos cualitativos
- /api/data/politicas - recomendaciones y metricas
- /api/data/contacto - configuracion del centro de contacto
- /api/data/admin - datos agregados del panel admin

6.2 API de administracion ('/api/admin/')

Requiere sesion con rol Autoridad:

- Gestion de espacios culturales (espacios, flujo de publicacion)
- Usuarios y roles (usuarios)
- Capas del mapa y sincronizacion (mapa-capas, capas)
- Plantillas de reportes (reportes-plantillas, reportes-config)
- Recursos cualitativos (recursos-cualitativos)
- Politicas y recomendaciones (politicas-recomendaciones, politicas-config)
- Consultas de contacto (consultas-contacto)
- Fuentes de informacion (fuentes)
- Logs de operacion (logs)

6.3 API publica v1 ('/api/v1/')

Expuesta en la seccion de contacto para integraciones externas:

- GET /api/v1/search - busqueda de espacios culturales
- GET /api/v1/espacios/geojson - espacios en formato GeoJSON
- GET /api/v1/layers/transporte - capa de transporte
- GET /api/v1/alcaldias/[id]/stats - estadisticas por alcaldia

Autenticacion opcional mediante token ('GEOARTE_API_TOKEN').

6.4 Exportaciones y reportes

- /api/reportes/generar - generacion de informes
- /api/reportes/descargar - descarga de archivos generados
- /api/politicas/export/brief y /informe - exportacion de politicas
- /api/investigacion/export/recurso - exportacion de recursos

7. Base de datos (Supabase / PostgreSQL)

El esquema se versiona en 'supabase/migrations/' con tablas principales para:

Area | Tablas / objetos
Usuarios | 'profiles' (rol, nombre, avatar)
Padron cultural | Espacios georreferenciados, tipos, estados de publicacion
Mapa territorial | 'territorio_geometria', 'metricas_alcaldia', 'capa_transporte_linea'
Sincronizacion | 'mapa_sync_log', funciones de recalcu de macrozonas
Reportes | Plantillas, configuracion, descargas ('export_downloads')
Investigacion | 'recursos_cualitativos' con indices de busqueda
Políticas | Recomendaciones, metricas y configuracion del centro
Contacto | 'consultas_contacto', configuracion del centro
Almacenamiento | Bucket 'avatars' para fotos de perfil

Las politicas RLS (Row Level Security) de Supabase controlan el acceso segun el rol del usuario autenticado.

8. Mapa territorial - operacion

8.1 Carga inicial (seed)

```
npm run seed:mapa
```

Carga poligonos de alcaldias y lineas de transporte desde GeoJSON hacia Supabase. Comandos individuales:

```
npm run seed:territorio  
npm run seed:transporte
```

8.2 Sincronizacion de metricas

```
npm run sync:mapa
```

Recalcula 'metricas_alcaldia' desde el padron y regenera macrozonas. Tambien disponible desde Admin -> Capas del mapa -> Sincronizar ahora.

8.3 Fuente de capa de transporte

Variable 'TRANSPORTE_CAPA_SOURCE':

- auto (por defecto) - GeoJSON empaquetado o Supabase segun disponibilidad
- geojson - solo archivos locales
- supabase - solo base de datos

9. Variables de entorno

Copiar '.env.example' a '.env.local':

Variable | Obligatoria | Descripcion

'NEXT_PUBLIC_SUPABASE_URL' | Si (prod) | URL del proyecto Supabase
'NEXT_PUBLIC_SUPABASE_ANON_KEY' | Si (prod) | Clave publica anonima
'NEXT_PUBLIC_SITE_URL' | Recomendada | URL del sitio (auth redirects)
'SUPABASE_SERVICE_ROLE_KEY' | Scripts/admin | Clave de servicio para seeds y panel
'NEXT_PUBLIC_ANIO_CORTE_METRICAS' | Opcional | Ano de corte de metricas
'TRANSPORTE_CAPA_SOURCE' | Opcional | Fuente de capa de transporte
'GEOARTE_API_TOKEN' | Opcional | Token de API publica v1

10. Scripts npm disponibles

Comando | Descripcion

'npm run dev' | Servidor de desarrollo en 'http://localhost:3000'
'npm run build' | Compilacion de produccion
'npm run start' | Servidor de produccion
'npm run lint' | Analisis estatico con ESLint
'npm run seed:mapa' | Carga geometrias y transporte a Supabase
'npm run sync:mapa' | Sincroniza metricas y macrozonas
'npm run diagnostico:padron' | Diagnostico del padron de espacios

11. Diseno y experiencia de usuario

- Tema claro/oscuro gestionado por ThemeProvider y ThemeScript (sin flash en carga).
 - Tokens de marca en CSS: navy institucional (--geo-navy), acento rosa (--geo-pink).
 - Layout responsivo con barra superior, busqueda cultural y pie de pagina.
 - Componentes compartidos: tarjetas KPI, tablas admin, modales de formulario, campos de busqueda.
-

12. Integracion con el ecosistema SECTEI

GEO ARTE Web comparte convenciones con la aplicacion movil Flutter del proyecto SECTEI:

- Mismos roles de perfil (Ciudadano, Investigador, Autoridad)
 - Mismas credenciales Supabase
 - Ano de corte de metricas alineado (ANIO_CORTE_METRICAS)
 - Padron georreferenciado de espacios culturales como fuente principal
-

13. Despliegue

La aplicacion es compatible con despliegue en Vercel (recomendado para Next.js) o cualquier hosting Node.js que soporte 'next start'.

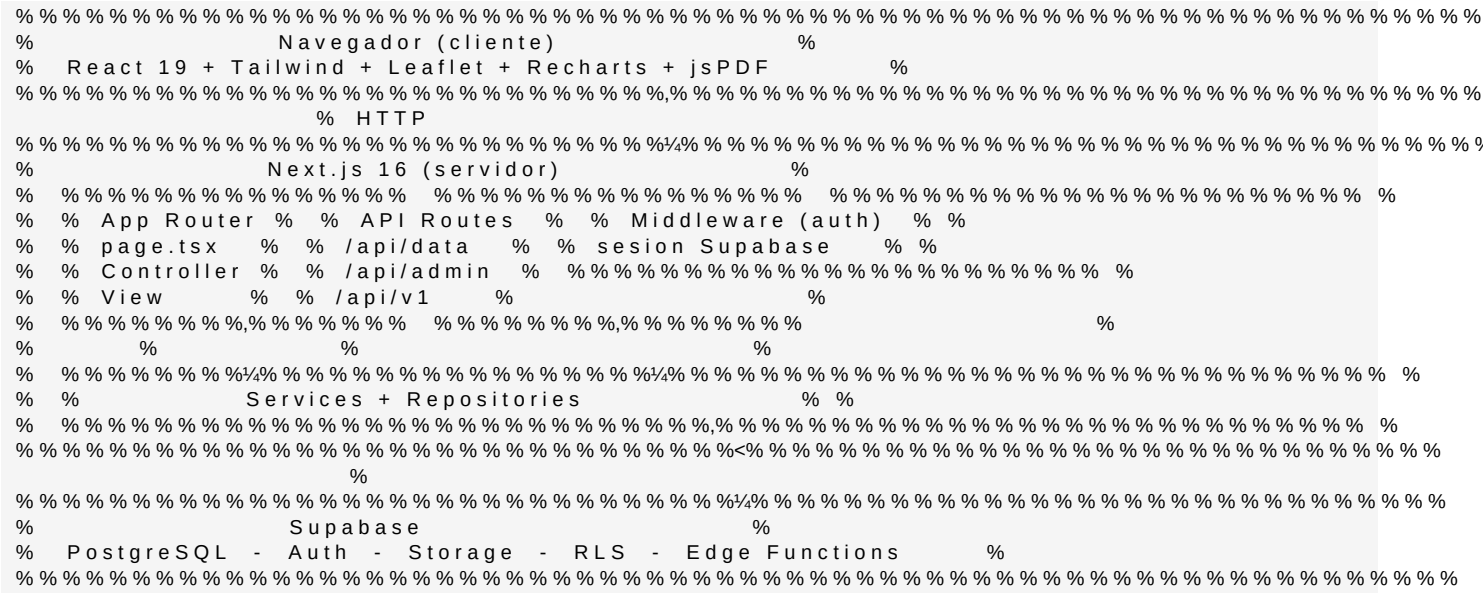
Requisitos para produccion:

1. Aplicar migraciones SQL en Supabase
 2. Configurar variables de entorno
 3. Ejecutar npm run seed:mapa en el primer despliegue
 4. Configurar URLs de redireccion en Supabase Auth
 5. Ejecutar npm run build y desplegar el artefacto
-

14. Documentacion relacionada

- Documento | Contenido
- 'docs/cliente/MANUAL-USUARIO.md' | Guia funcional para ciudadanos, investigadores y autoridades
 - 'docs/tecnico/ARQUITECTURA-DESARROLLO.md' | Patron MVC y convenciones de codigo
 - 'docs/tecnico/OPERACION-MAPA.md' | Seeds, sincronizacion y capas del mapa
 - 'docs/cliente/PANEL-ADMINISTRACION.md' | Panel de administracion
 - 'docs/cliente/CONTROL-DE-CAPAS-MAPA.md' | Gestion de capas del mapa territorial
 - 'docs/tecnico/INSTALACION.md' | Requisitos e instalacion local
 - 'docs/tecnico/DESPLIEGUE.md' | Despliegue en produccion
 - 'README.md' | Inicio rapido del proyecto

15. Diagrama de arquitectura



Documento generado para el equipo de desarrollo e integracion de GEO ARTE CDMX.